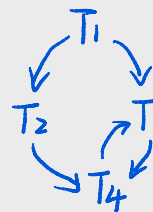


先读中介  
再用先读后写。

三、[10 分] 根据如下所示的调度 S 判断是否为冲突可串行化调度？如果不是，请分析原因。是否为视图可串行化调度，如果不是，请分析原因？如果是，请给出视图可串行化调度顺序。

| $T_1$    | $T_2$    | $T_3$    | $T_4$    |
|----------|----------|----------|----------|
| Read(Z)  |          |          |          |
| Write(Z) |          |          |          |
| Read(X)  |          |          |          |
|          | Write(Z) |          |          |
| Write(X) |          |          |          |
|          |          |          | Read(Z)  |
|          |          |          | Write(Z) |
|          |          |          | Read(X)  |
|          |          | Read(X)  |          |
|          |          | Write(X) |          |
|          |          |          | Write(X) |



有环，不是冲突可串行化。

假设视图可串行化。

$T_1 > T_2 > T_4$ . 对 Z.

$T_1 > T_3$ . 对 X.

$T_1$  最先,  $T_4$  最后.

$\langle T_1, T_2, T_3, T_4 \rangle$

$\langle T_1, T_3, T_2, T_4 \rangle$ .

若矛盾，  
则视图不可  
串行化。

答案：视图可串行化  $T_1-T_2-T_3-T_4$

四、[16 分] 设有属于 1NF 的关系模式  $R = (A, B, C, D, E)$ ， $R$  上的函数依赖集

$F = \{ E \rightarrow C, A \rightarrow BE, B \rightarrow E, BD \rightarrow C, AB \rightarrow E \}$ 。

1、给出关系模式  $R$  的候选码。

2、关系模式  $R$  是否满足 BCNF，如果不满足给出理由？根据  $F$  中给出的函数依赖次序，根据 BCNF 分解算法对关系模式  $R$  进行分解。要求给出步骤。

3、关系模式  $R$  是否满足 3NF，如果不满足给出理由？根据 3NF 分解算法对关系模式  $R$  进行分解，要求给出步骤。

五、[46 分] 设有如下关系表，其中下划线表示主键

- $branch = (\underline{branch\_name}, branch\_city, assets)$
- $customer = (\underline{customer\_id}, customer\_name, customer\_street, gender)$
- $loan = (\underline{loan\_number}, amount, branch\_name)$
- $account = (\underline{account\_number}, balance, branch\_name)$
- $borrower = (\underline{customer\_id}, \underline{loan\_number})$
- $depositor = (\underline{customer\_id}, \underline{account\_number})$

(一)、以数据定义语言定义上述关系表，表明必要的主键、外键和 check 约束( $assets > 0$ ;  $balance > 0$ ,  $amount > 0$ )。(6 分)

(二)、以关系代数表达式完成以下功能 (14 分，每小题 2 分)

1、查询位于 Brooklyn 城市，并且资产大于 100000 的支行名称和存款账号数量。

2、客户编号为 001 的客户在 Brooklyn 城市多个支行有贷款账户，查询在所有这些支行中也均有贷款帐户的客户编号。↵

3、在数据库中加入如下信息：为客户编号为 001974 的客户在 Perryridge 支行建立存款帐户，帐户编号为 A-973，帐户额为 1200。↵

4、将贷款编号为 L-2078 的贷款信息从数据库中删除。↵

5、将贷款编号为 L-2078 的贷款额度修改为 1000。↵

6、查询贷款额度的最大值。↵

7、找到在 Perryridge 支行有贷款的客户的名字。↵

（三）、以 SQL 语句完成（二）中要求的功能，其中查询语句要求用一条 SQL 语句实现（14 分，每小题 2 分，多个 SQL 表达式注明执行顺序）：↵

1、查询在存款账号数量多于 1000（不包含 1000），并且名称以 '%CH' 结尾的支行名称↵

2、编号为 001 的客户在 Brooklyn 城市多个支行有贷款账户，查询在所有这些支行中也均有贷款帐户的客户编号，结果按照降序排列↵

3、在数据库中加入如下信息：为编号为 A-973 的客户在 Perryridge 支行建立存款帐户，帐户编号为 001974，帐户额为 1200↵

4、将客户编号为 002078 的顾客的所有相关信息从数据库中删除。↵

5、查询账户存款余额平均值大于 \$1200，且客户人数多于 100 人的支行名称↵

6、将贷款编号 L2078 变更为 L2763。↵

7、查询资产高于 Brooklyn 城市所有支行资产的支行名称。↵

（四）以 SQL 语句完成以下功能（12 分，每小题 3 分）↵

1、修改表结构：从关系表 *customer* 删除字段 *gender*，并加上 *customer\_city* 列。↵

2、建立角色 *Teller*，分别将查询 *branch* 表、更新 *account* 表以及在 *depositor* 的所有操作权限授予给 *Teller*。将更新 *account* 表的权限从 *Smith* 收回。↵

3、建立触发器：银行对存款账户透支采取如下措施。1) 将存款账号余额清零；2) 以存款账号建立贷款账号，以透支数额为已经贷款数额。↵

4、建立视图 *dep\_borr*，根据该视图可以查询没有贷款账户并且至少有 1 个存款账户的客户编号和姓名。↵

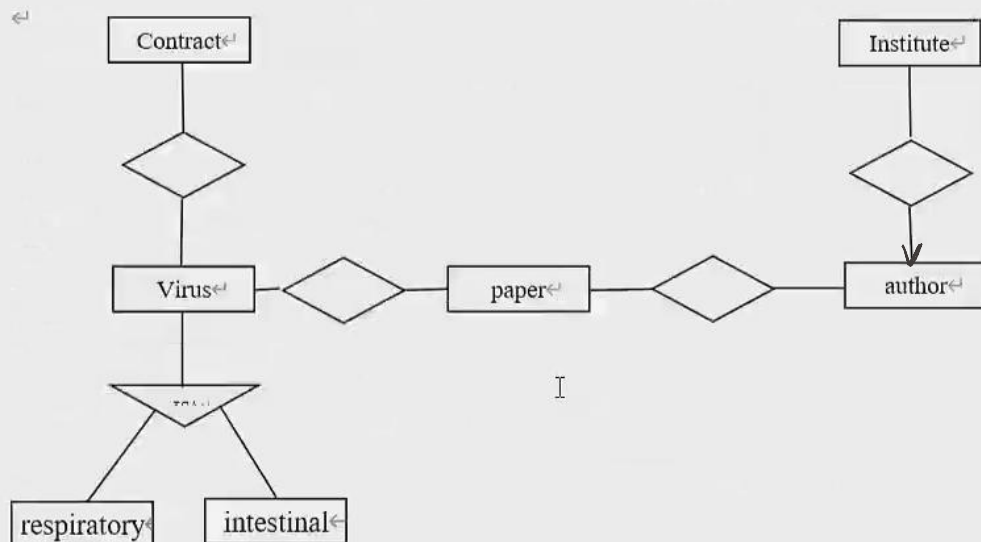
## 2020 级《数据库系统原理》考试试题答案 (A 卷)

一、[12 分] 单项选择题 (每小题 1 分)

1. A. 2. D. 3. A. 4. D. 5. C. 6. A. 7. A. 8. A. 9. C. 10. C. 11. C. 12. A.

二、[16 分] 根据如下关系描述 (其中下划线表示主键) 绘制 ER 图。将 ER 图转换成对应的关系表, 并以数据定义语言定义上述关系表, 表明必要的主键、外键和 check 约束。

一个医学研究小组维护关于某些病毒的所有已出版论文 Paper 的数据库。关于病毒 Virus 所记录的信息 (包括其学名 Virus\_Name 和一个自由文本 Description 格式的描述)。介绍病毒的论文可以有一名或多名作者 Author, 而且可以涉及一种或多种病毒, 内容包括 (论文摘要 Abstract 和论文正文 Content)。完成论文的科研作者隶属于某个研究机构 Institute。作者的信息包括 (工作证号 Author\_id、作者名字 Author\_name、出生日期 DateofBirth、年龄 Age 和联系方式 Contact\_inf)。研究机构的信息包括 (机构名称 Iname、地址 Address 和类型 Type)。根据传染途径, 论文将病毒分为 (呼吸道病毒 (respiratory) 和肠道病毒 (intestinal))。呼吸道病毒需要记录发烧症状 (Symptom), 肠道病毒需要记录倦怠症状 (lassitude), 还需保存与病毒有关的研究合同 Contract 的细节 (合同号 Contract#、金额 Amount、开始日期 Start\_date 和结束日期 End\_date 等)。



三、[10 分] 根据如下所示的调度 S 判断是否为冲突可串行化调度？如果不是，请分析原因。是否为视图可串行化调度，如果不是，请分析原因？如果是，请给出视图可串行化调度顺序。

| $T_1$    | $T_2$    | $T_3$    | $T_4$    |
|----------|----------|----------|----------|
| Read(Z)  |          |          |          |
| Write(Z) |          |          |          |
|          | Read(Z)  |          |          |
| Read(X)  |          |          |          |
|          | Write(Z) |          |          |
| Write(X) |          |          |          |
|          |          |          | Read(Z)  |
|          |          |          | Write(Z) |
|          |          |          | Read(X)  |
|          |          | Read(X)  |          |
|          |          | Write(X) |          |
|          |          |          | Write(X) |

答案：视图可串行化  $T_1-T_2-T_3-T_4$

四、[15 分] 设有属于 1NF 的关系模式  $R = (A, B, C, D, E)$ ， $R$  上的函数依赖集

$F = \{E \rightarrow C, A \rightarrow BE, B \rightarrow E, BD \rightarrow C, AB \rightarrow E\}$ 。

1、给出关系模式  $R$  的候选码。

2、关系模式  $R$  是否满足 BCNF，如果不满足给出理由？根据  $F$  中给出的函数依赖次序，根据 BCNF 分解算法对关系模式  $R$  进行分解。要求给出步骤。

3、关系模式  $R$  是否满足 3NF，如果不满足给出理由？根据 3NF 分解算法对关系模式  $R$  进行分解，要求给出步骤。

答案：AD

不满足，BCNF 分解算法：{C, E} {A, B, E} {A, D} (4 分)

不满足，3NF 分解算法：{A, B} {B, E} {E, C} {A, D} (6 分)

五、[47 分] 设有如下关系表，其中下划线表示主键

- $branch = (\underline{branch\_name}, branch\_city, assets)$
- $customer = (\underline{customer\_id}, customer\_name, customer\_street, gender)$
- $loan = (\underline{loan\_number}, amount, \underline{branch\_name})$
- $account = (\underline{account\_number}, balance, \underline{branch\_name})$
- $borrower = (\underline{customer\_id}, \underline{loan\_number})$
- $depositor = (\underline{customer\_id}, \underline{account\_number})$

(一)、以数据定义语言定义上述关系表，表明必要的主键、外键和 check 约束。(6 分)

```
Create table branch
(branch_name varchar(20),
branch_city varchar(20),
assets integer,
Primary key (branch_name),
Check asset>0)
```

I

```
Create table customer
(customer_id char(20),
customer_name varchar(20),
customer_street varchar(20),
Gender char(2),
Primary key (customer_id))
```

```
Create table loan
(loan_number char(20),
amount numeric(10,2),
branch_name varchar(20),
```

```
Primary key (loan_number)
Foreign key (branch_name) references branch))
```

```
Create table account
(account_number char(20),
balance varchar(20),
branch_name varchar(20),
```

I

```
Primary key (account_number)
Foreign key (branch_name) references branch))
```

```
Create table borrower
(customer_id char(20),
loan_number char(20),
Primary key (customer_id, loan_number)
Foreign key (customer_id) references customer,
```

Foreign key (loan\_number) references loan)↵

↵

Create table depositor = (customer\_id, account\_number)↵

(customer\_id char(20),↵

account\_number char(20),↵

Primary key (customer\_id, account\_number)↵

Foreign key (customer\_id) references customer,↵

Foreign key (account\_number) references account)↵

↵

(二)、以关系代数表达式完成以下功能(14分, 每小题2分, 第6小题4分)↵

1、查询位于 Brooklyn 城市, 并且资产大于 100000 的支行名称和存款账号数量。↵

$\Pi_{branch\_name} \sigma_{Count(account\_number) (\sigma_{branch\_city = 'Brooklyn' \wedge assets > 10000} (branch \bowtie account))}$ ↵

↵

2、编号为 001 的客户在 Brooklyn 城市多个支行有贷款账户, 查询在所有这些支行中也均有贷款帐户的客户编号。↵

$\Pi_{customer\_id, branch\_name} \sigma_{branch\_city = 'Brooklyn'} (branch \bowtie loan \bowtie borrower) \div$ ↵

$\Pi_{branch\_name} \sigma_{branch\_city = 'Brooklyn', customer\_id = '001'} (branch \bowtie loan \bowtie borrower)$ ↵

↵

↵

3、在数据库中加入如下信息: 为客户编号为 001974 的客户在 Perryridge 支行建立存款帐户, 账户编号为 A-973, 帐户额为 1200。↵

$account \leftarrow account \cup \{('A-973', 1200, 'Perryridge')\}$ ↵

$depositor \leftarrow depositor \cup \{('A-973', '001974')\}$ ↵

4、将贷款编号为 L-2078 的贷款信息从数据库中删除。↵

$borrower \leftarrow borrower - \sigma_{loan\_number = 'L-2078'} (borrower)$ ↵

$loan \leftarrow loan - \sigma_{loan\_number = 'L-2078'} (loan)$ ↵

↵

5、将贷款编号为 L-2078 的贷款额度修改为 1000。↵

$loan \leftarrow loan - \sigma_{loan\_number = 'L-2078'} (loan)$ ↵

$\cup \Pi_{loan\_number, 1000, branch\_name} \sigma_{loan\_number = 'L-2078'} (loan)$ ↵

6、查询贷款额度的最大值。↵

$\Pi_{amount(loan)} - \Pi_{loan.amount} \sigma_{loan.amount < d.amount (loan \times \rho_d(loan))}$ ↵

7、找到在 Perryridge 支行有贷款的客户的名字。↵

$\Pi_{customer\_name} (\sigma_{branch\_name = 'Perryridge'} ($   
 $\sigma_{borrower.loan\_number = loan.loan\_number} (borrower \times loan)))$ ↵

(三)、以 SQL 语句完成 (二) 中要求的功能, 其中查询语句要求用一条 SQL 语句实现 (14 分, 每小题 2 分):

1、查询存款账号数量多于 1000 (不包含 1000), 并且名称以 '%CH' 结尾的支行名称

```
Select branch_name, count(account_number)
From account
Where branch_name like '%\%CH' escape '\
Having count(account_number)>10000
```

2、编号为 001 的客户在 Brooklyn 城市多个支行有贷款账户, 查询在所有这些支行中也均有贷款帐户的客户编号, 结果按照降序排列

```
Select customer_id
From customer A
Where not exists
    (select branch_name from loan, borrower, customer, branch where
    loan.loan_number=borrower.loan_number and
    borrower.customer_id=customer.customer_id and
    loan.branch_name=branch.branch_name and
    customer.customer_id='001' and branch.branch_city='Brooklyn'
    except
    select branch_name from loan, borrower, customer, branch where
    loan.loan_number=borrower.loan_number and
    borrower.customer_id=customer.customer_id and
    loan.branch_name=branch.branch_name and
    branch.branch_city='Brooklyn' and B.customer_id=A.customer_id)
Order by customer_id desc
```

3、在数据库中加入如下信息: 为编号为 A-973 的客户在 Perryridge 支行建立存款帐户, 帐户编号为 001974, 帐户额为 1200

4、将客户编号为 002078 的客户的的所有相关信息从数据库中删除。

```
Delete from depositor where customer_id='002078'
```